



Toward a framework for self-adaptive workflows in cyber-physical systems

Ronny Seiger¹ · Steffen Huber¹ · Peter Heisig¹ · Uwe Aßmann¹

Received: 19 December 2016 / Revised: 10 October 2017 / Accepted: 16 November 2017 / Published online: 24 November 2017
© Springer-Verlag GmbH Germany, part of Springer Nature 2017

Abstract

With the establishment of Cyber-physical Systems (CPS) and the Internet of Things, the virtual world of software and services and the physical world of objects and humans move closer together. Despite being a useful means for automation, BPM technologies and workflow systems are yet not fully capable of executing processes in CPS. The effects on and possible errors and inconsistencies in the physical world are not considered by “traditional” workflow engines. In this work we propose a framework for self-adaptive workflows in CPS based on the MAPE-K feedback loop. Within this loop monitoring and analysis of additional sensor and context data is used to check for unanticipated errors in the physical world. Planning and execution of compensation actions restores *Cyber-physical Consistency*, which leads to an increased resilience of the process execution environment. The framework facilitates the separation of CPS aspects from the “regular” workflow views. We show the feasibility of this approach in a smart home scenario and discuss the application of our approach for legacy BPM systems.

Keywords Workflows for the Internet of Things · Cyber-physical Systems · Self-adaptive Workflows · Real-world processes · Cyber-physical consistency

1 Introduction

The application of Business Process Management (BPM) and workflow technologies in the new areas of the Internet of Things (IoT) and Cyber-physical Systems (CPS) may lead to increased automation, higher efficiency, better resource utilization, and increased comfort for users as well as higher product qualities. With the advancement of these technologies the BPM community faces novel challenges resulting from the adaptation of workflows to interact with real-world objects, sensors, actuators, humans and digi-

tal services [30]. These new classes of systems rely heavily on this interaction between the physical and the cyber worlds. In contrast to organizational BPM application fields, automating processes in process-aware cyber-physical information systems requires the consideration of processes’ real-world effects and context [52] as well as a certain level of adaptivity and autonomy to cope with unanticipated situations [15].

In CPS various real-world factors may jeopardize the successful process execution. Solely relying on feedback provided by built-in sensors and software components of CPS devices for evaluating the outcome of task executions is usually not sufficient enough to provide a consistent view of the virtual and the physical world. Unexpected failures in the real world (e.g., malfunctioning actuators, physical obstacles, the imprecise positioning of an object, or a lack of resources) may not be detected by the respective hardware. With more and more sensors and embedded computers available for building the IoT, the feedback from execution engines and control software can be augmented by additional real-world sensor and context information. These new information sources enable a more reliable verification of the process execution to detect errors, eval-

Communicated by Dr. Ilia Bider and Rainer Schmidt.

✉ Ronny Seiger
Ronny.Seiger@tu-dresden.de
Steffen Huber
Steffen.Huber@tu-dresden.de
Peter Heisig
Peter.Heisig@tu-dresden.de
Uwe Aßmann
Uwe.Assmann@tu-dresden.de

¹ Institute of Software and Multimedia Technology, Technische Universität Dresden, 01062 Dresden, Germany

uate changes within the physical world, and relate these changes to the execution. In case of errors, deviations and inconsistencies, the process has to be adapted by executing compensating actions, which can be evaluated again by sensors. This feedback loop enables autonomy and self-adaptation for processes and engines in CPS. However, current approaches mostly focus on extending “traditional” business processes and BPM systems into the physical world or on increasing the adaptivity of virtual BPM systems neglecting the *physical* aspects. The issue of preserving consistency between *cyber* and *physical* world process execution by applying *Feedback Loops* and incorporating additional information sources has not been discussed in detail yet.

In this work, we present an approach for enabling self-adaptive workflows based on the MAPE-K (Monitor, Analyze, Plan and Execute on a Knowledge Base) control loop for self-adaptive systems [6]. In addition to feedback from the process management system and services, sensor data from physical devices are used to monitor and analyze the real-world effects of process activity executions. If an inconsistency between the sensed *physical world* and the assumed *cyber world* can be detected, a compensation strategy is chosen and the adapted process is executed. That way, we achieve a certain level of self-healing for CPS workflows. We propose to use this MAPE-K loop as a generic framework for existing BPM systems to extend their functionality toward managing cyber-physical workflows [38,40].

The paper is structured as follows: Sect. 2 presents an application scenario for cyber-physical workflows in the smart home domain. Section 3 introduces the concept of *Cyber-physical Consistency* and the MAPE-K loop for processes. Section 4 presents a framework to enable self-adaptive workflows. Section 5 evaluates and discusses our approach based on the scenario. Section 6 reviews related research. Section 7 concludes the paper and shows starting points for future work.

This paper is an extended version of the contribution [38] published as part of the proceedings of the BPMDS 2016 working conference. It has been restructured, the introduction has been partially rewritten. The *Smart Lighting* scenario has been extended with a more detailed presentation and discussion of the experiments and results. A more formal definition for the terms *Cyber-physical Consistency* and *Consistency Levels* has been added. The application of the MAPE-K feedback loop at the process modeling and execution level is proposed as a general framework for existing process engines. The related work section is restructured and contains a more thorough discussion and comparison with related approaches.

2 Smart lighting scenario

The scenario is built around smart home environments but can also be relevant for smart factories. Having enough light for working is one of the prerequisites for people to keep focused and maintain a good eyesight. A constant room illumination may also be required in production environments to achieve certain quality levels for products (e.g., in photolithography). A business process shall be used to realize a light control loop, which maintains a constant illumination level for a specific room. To increase energy efficiency the lighting should be decreased in case a specified threshold is exceeded. The corresponding process model is shown using a more technical-oriented modeling notation for workflows in CPS [41] in Fig. 1. With events and REST services being basic building blocks for IoT and CPS [14,45] this notation supports CPS workflow specifications that can be executed by our PROtEUS system [39].

Upon instantiation the first process step calls a REST Web service to activate the light switch. The *TriggeredEvent* process step listens for changes from light sensors (defined in an EPL pattern [41]). If the illumination levels are below 650 lux and above 750 lux, the succeeding process step is triggered. This conditional step (*If*) decides about an increase or decrease of the current illumination based on the sensor data from the previous step. In case it is too bright, a REST service will be invoked to dim the light switch. Otherwise its lighting levels will be increased via the REST service. This whole subprocess (*LightControlLoop*) is executed in a loop to enable a continuous process-based control of the room illumination once the light has been switched on.

Within this scenario various obstacles and errors may jeopardize the successful process execution that may not be detected by the process engine or by the software controlling the light switch—in our case a HomeMatic dimmer switch, which is not able to detect issues with plugged in devices. For example, the light bulb might be broken or wearing off, which leads to the process trying to increase the brightness by powering up the dimmer switch without ever reaching the desired threshold. This increases energy consumption and reduces work place comfort. This is an example for an error that cannot be detected by the respective CPS device as the manufacturer cannot anticipate every possible error case, which is why we suggest to consider information from additional sources to verify the execution.

3 Cyber-physical Consistency for processes

The scenario described in the previous section illustrates some of the new challenges that come along with the ongoing trend toward *Cyber-physical Convergence* [7] in modern information systems, which leads to the emergence of the

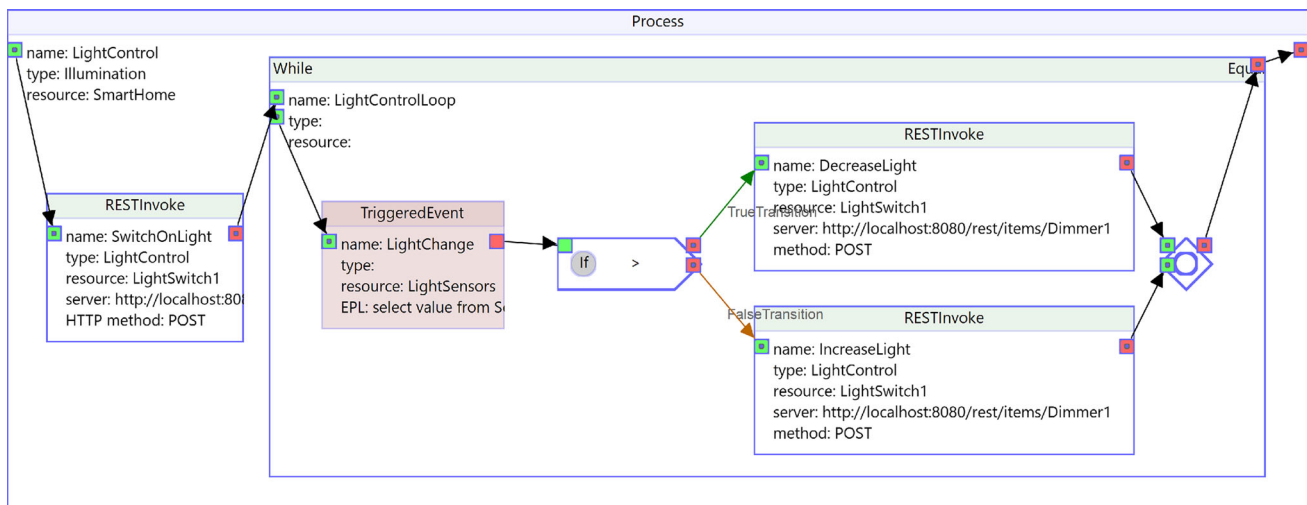
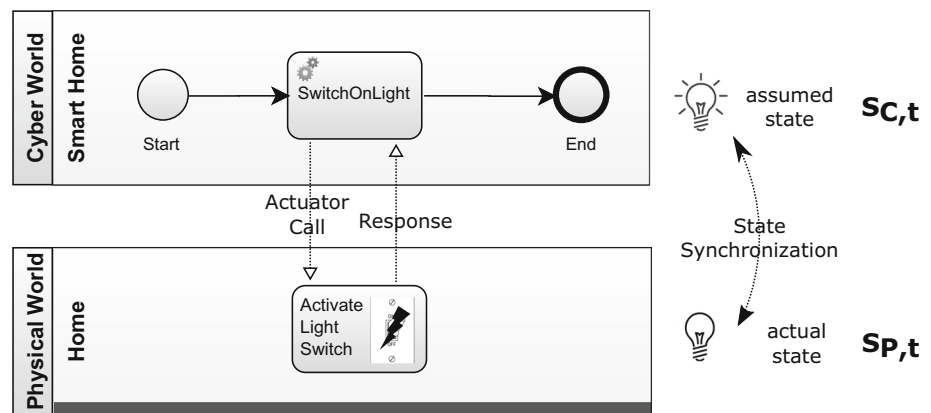


Fig. 1 Smart Lighting scenario process

Fig. 2 Synchronization between cyber world and physical world states



new class of *Cyber-physical Systems* (CPS), i.e., systems consisting of mutually-influencing digital and physical components [26]. Traditional BPM systems rely on service responses and process related events triggered by the workflow engine to verify the execution. Relying on built-in error detection mechanisms of BPM systems and control applications proved feasible for the verification of purely virtual business processes. When extending the concept of business processes to also support *CPS workflows* [39], monitoring and analysis capabilities of BPM systems have to be augmented to reflect and check the cyber-physical effects of processes. The emergence of new sensor types and “smart” devices from IoT developments facilitates the extension of existing devices and workflow systems with new information sources to detect new error situations.

As an extension of the well-known ACID criteria [13], we introduce the notion of *Cyber-physical Consistency CPC* for a workflow in CPS as a matching of its assumed physical state (i.e., virtual state) and its actual physical state measured by sensors after execution. Cyber-physical consistency can be observed if the workflow’s virtual state is in sync with

the workflow’s real-world state. Figure 2 illustrates the challenge of synchronizing the virtual and real world related to the *Smart Lighting* scenario: the process is supposed to switch on the light in a certain room by issuing a service call to the switch’s control software. The software reports back the successful execution and the process instance finishes assuming the light is activated. However, a burnt light bulb leads to an undesired physical state, which cannot be detected by the actuator’s built-in control software and therefore cyber-physical consistency is *violated*.

3.1 Physical process context

To have a more formal representation of the concept of *Cyber-physical Consistency (CPC)* we denote $C_t = \{c_{1,t}, \dots, c_{n,t}\}$ as a set of all physical context factors $c_{i,t}$ at a given point in time t . A context factor is defined as a tuple $c_t = (n, v_t)$, where $n \in N$ represents a unique identifier (e.g., *Kitchen-Light*) from a set N of identifiers and $v_t \in V_t$ represents a value from a set V_t of context values at a given point in time t (e.g., 700 lux). Values can be numerals for measurable

properties but also more abstract concepts, e.g., execution states or locations. We define $S_{P,t} \subseteq C_t$ as a set of all physical context factors that are manipulated by or relevant for the execution of a specific process (step). $S_{P,t}$ represents the *actual* (physical) *state* of the process execution at a given point in time t .

3.2 Virtual process context

Corresponding to the previous definition we denote $C'_t = \{c'_{1,t}, \dots, c'_{n,t}\}$ as a set of all virtual representations of physical context factors $c'_{i,t}$ at a given point in time t (*assumed state* of the physical world). A virtual context factor is defined as a tuple $c'_t = (n', v'_t)$, where $n' \in N$ represents a unique identifier and $v'_t \in V_t$ represents a value at a given time t . We define $S_{C,t} \subseteq C'_t$ as a set of all virtual context factors that are manipulated by or relevant for process execution. $S_{C,t}$ is the *assumed* (virtual) *state* of the process execution at a given point in time t .

3.3 Cyber-physical Consistency

From that we derive the following condition for the process execution being in a state of cyber-physical consistency (CPC_t) at a given point in time t (i.e., for all relevant context factors $c_t \in S_{P,t}$ of the actual physical state, there exists a corresponding virtual context factor $c'_t \in S_{C,t}$ of the assumed physical state where $n = n'$ and $v_t = v'_t$):

$$CPC_t \Leftrightarrow \forall c_t : c_t \in S_{P,t}, \exists c'_t : c'_t \in S_{C,t} \text{ where } c_t \equiv c'_t \text{ with } c_t \equiv c'_t : (n = n') \wedge (v_t = v'_t) \quad (1)$$

Analogous to that we define the condition for the process execution not being in a state of cyber-physical consistency (i.e., there exists at least one context factor $c_t \in S_{P,t}$ of the actual physical state that has a corresponding virtual context factor $c'_t \in S_{C,t}$ of the assumed physical state where $n = n'$ and $v_t \neq v'_t$):

$$\neg CPC_t \Leftrightarrow \exists c_t : c_t \in S_{P,t}, \exists c'_t : c'_t \in S_{C,t} \text{ where } c_t \not\equiv c'_t \text{ with } c_t \not\equiv c'_t : (n = n') \wedge (v_t \neq v'_t) \quad (2)$$

4 Framework for self-adaptive workflows

In this section we present a framework for self-adaptive workflows based on the MAPE-K feedback loop [9], which is used to detect and repair cyber-physical inconsistencies. This framework is intended to be applicable to process engines in the context of CPS (e.g., the PROtEUS system [18,39])

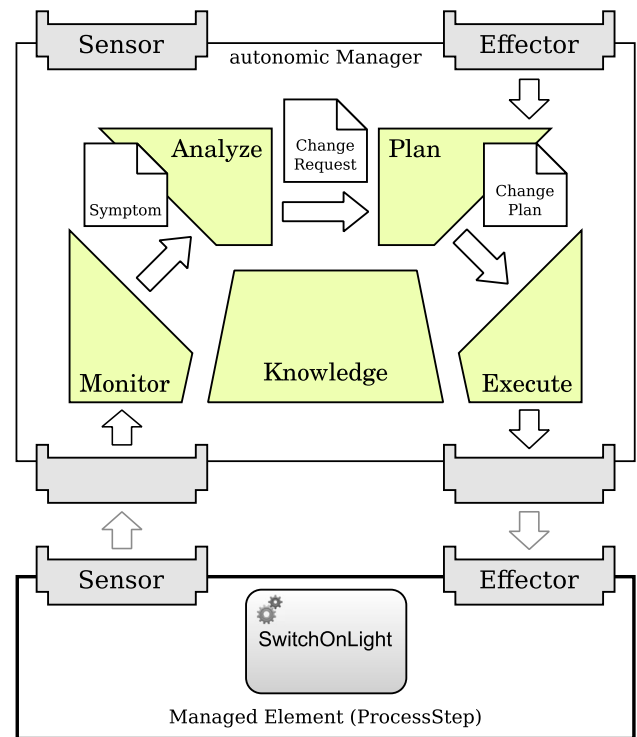


Fig. 3 Managing process steps with MAPE-K loops in accordance with [20]

or even as a means to retrofit existing service-based BPM systems to add CPS workflows and self-adaptation to their capabilities.

4.1 MAPE-K control loops for ensuring CPS consistency

In order to evaluate and ensure cyber-physical consistency for CPS workflows, we will apply the concept of the MAPE-K feedback loop known from the engineering of self-adaptive systems area [6] to process execution. This loop consists of the phases *Monitor* (M), *Analyze* (A), *Plan* (P) and *Execute* (E)—all repeatedly executed on the *Knowledge Base* (K). Figure 3 shows our adaptation of the MAPE-K concept proposed by IBM [20]. We regard the process/process step in question and its executing resource as *Managed Element* (here: *SwitchOnLight* task from Fig. 2). In the *Monitor* phase, we collect sensor data from the physical world. Relevant changes within these data (called *Symptoms*) are then analyzed to check for cyber-physical consistency according to its definition from Sect. 3 to correlate changes in the real world to the process. In case an inconsistency within one of the context factors is detected, a change request is triggered and a compensation strategy (*Change Plan*) is chosen and executed to try to restore CPC and continue with process execution as planned. The MAPE-K loop may be executed repeatedly to verify the success of the compensating action(s).

4.2 Modeling self-adaptive CPS workflows

To define the physical effects of a workflow or an activity, respectively, “traditional” process meta-models from the BPM domain (e.g., BPMN and WS-BPEL) have to be extended with additional information as they do not support the specification of the effects or outcome of task executions. They have been designed to model business processes in organizational contexts, which produce data, events or messages but not specify effects on the real world. Figure 4 shows our proposal for extending process meta-models with specific attributes regarding their influence on the real world in UML notation. We describe the extension with respect to the exemplary component-based process metal-model for CPS presented in [41] and used in our application scenario.

The *CpsStep* interface has to be implemented by the abstract class *ProcessStep*, which represents either an atomic process activity (cf. *Activity* class in BPMN [49]) or a composite process step (i.e., a process or a subprocess containing multiple process steps). This interface adds the new attribute *cyberPhysical* to the process step. With this attribute the process modeller specifies if the step has an effect on the real. In case this is true, a *Goal* can be defined that will be used for analysis and adaptation in the MAPE-K loop. This goal has to be fulfilled to confirm cyber-physical consistency. We decided to use a more declarative goal-oriented approach for defining the domain-specific outcome as proposed in [22], because modeling every possible outcome and error as well as corresponding compensations is not feasible for complex CPS systems [23].

The *Goal* class may subsume several *Objectives* that have to be met individually for the overall goal to be fulfilled. The *Objective* class contains the following attributes. The exam-

ples describe the goal “Enough light for working” including the objective that after triggering a light control switch via a process, the light levels in the kitchen should be above 700 lux within 5 s. Otherwise, a malfunction is assumed and a compensation has to be searched in the *Plan* phase.

4.2.1 Context path

The context path defines the relevant context measuring points from the actual state $S_{P,t}$ to be monitored (e.g., specific sensor and process data) in the *Monitor* phase. We base our concept on a graph representation of a context ontology [19], which acts as the *Knowledge Base K*. Therefore, this attribute defines one or more paths within this context graph leading to specific sensor nodes (cf. Sect. 4.3). An exemplary context path representing the value of a light sensor (*lightIntensity*) in the kitchen could look like this (written using the *Cypher* language [46]):

```
MATCH (kitchen)-[:instanceOf]->(room)
MATCH (light)-[:instanceOf]->(sensor)
MATCH (light)-[:isIn]->(kitchen)
RETURN light.value AS lightIntensity
```

4.2.2 Satisfied assertion

The satisfied assertion defines the required change within relevant context data as result of the execution of the process step. In the *Analyze* phase, this attribute is used to evaluate the success of the operation, i.e., to compare assumed state $S_{C,t}$ with actual state $S_{P,t}$. The objective is satisfied if the Boolean assertion is evaluated to be *true*. An exemplary sat-

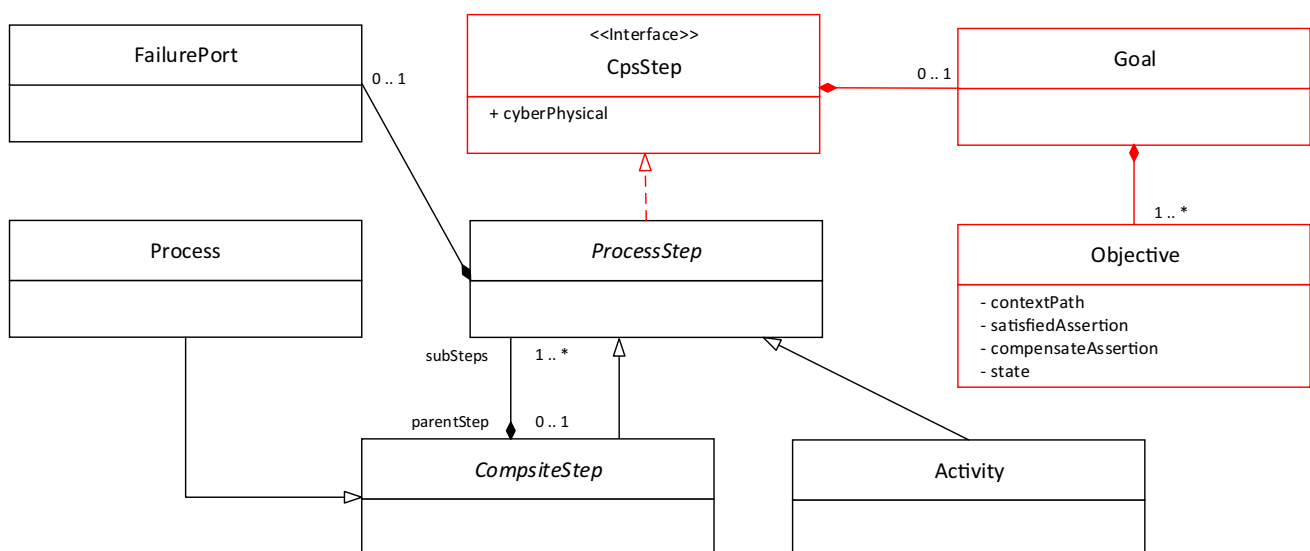


Fig. 4 Process meta-model extensions for self-adaptive CPS workflows

sified assertion requiring the light intensity to be above a certain threshold (700 lux) could look like this:

```
#lightIntensity > 700
```

4.2.3 Compensation assertion

The compensation assertion defines a condition for initiating the *Plan* phase due to some unexpected behavior. This Boolean expression may include context and other data (e.g., a certain time frame), which is used to evaluate the need for finding a compensation action in the *Plan* phase). An exemplary compensation assertion testing if an objective was created more than 5 s ago could look like this (i.e., after a maximum of 5 s there should be a significant change within the light levels):

```
#objective.created.isBefore(#now.minusSeconds(5))
```

4.2.4 State

The *State* property is used as a runtime attribute to represent the state of an objective (satisfied, unsatisfied, compensation needed, or failed). For the goal to be fulfilled—and with it the related process step to be executed successfully—all its objectives have to be of state *satisfied*. An objective is *unsatisfied* if neither the satisfied assertion nor the compensation assertion is true. If the compensation assertion is true, then a *compensation* is *needed*. In case no compensation can be found during the *Plan* phase and the loop finishes without fulfilling the objective, it is in state *failed*.

To cope with this case that the execution of the MAPE-K feedback loop is not able to find compensations for errors and therefore to fulfill all objectives (objective state is *failed*), we extend the meta-model from [41] with the special port class *FailurePort*. This port activates an explicitly modeled process branch as a fallback solution to handle unresolvable errors (e.g., trigger a *Human Task* to ask for user interactions [39,40]). Other process engines/notations may be extended in a similar way to use failure branches or other built-in means for error handling.

4.3 Knowledge base

The knowledge base holds data about the physical and virtual context of the CPS based on sensor data, actuators and processes. It is updated with every change within these data and accessed by all software components. In addition to represent information about the processes and their context as proposed in [37], we use a graph-based representation of the DogOnt ontology model from domotic environments [4] to describe sensors, actuators and their relations for our applications in the smart home [19]. Figure 5 shows an extract from the context model describing the properties of an instance of (*Dimmer2*) of class *DimmerSwitch*. The central top level class in the knowledge base used to describe the CPS devices is the *Item* class. The *Dimmer1* and *Dimmer2* objects are instances of type *DimmerSwitch*, and they are located in the *Kitchen*. Besides that a Dimmer object has a particular state (*State_Dimmer2*) (here: the current power level from 0 to 100%). A description of the dimmer's functionality is

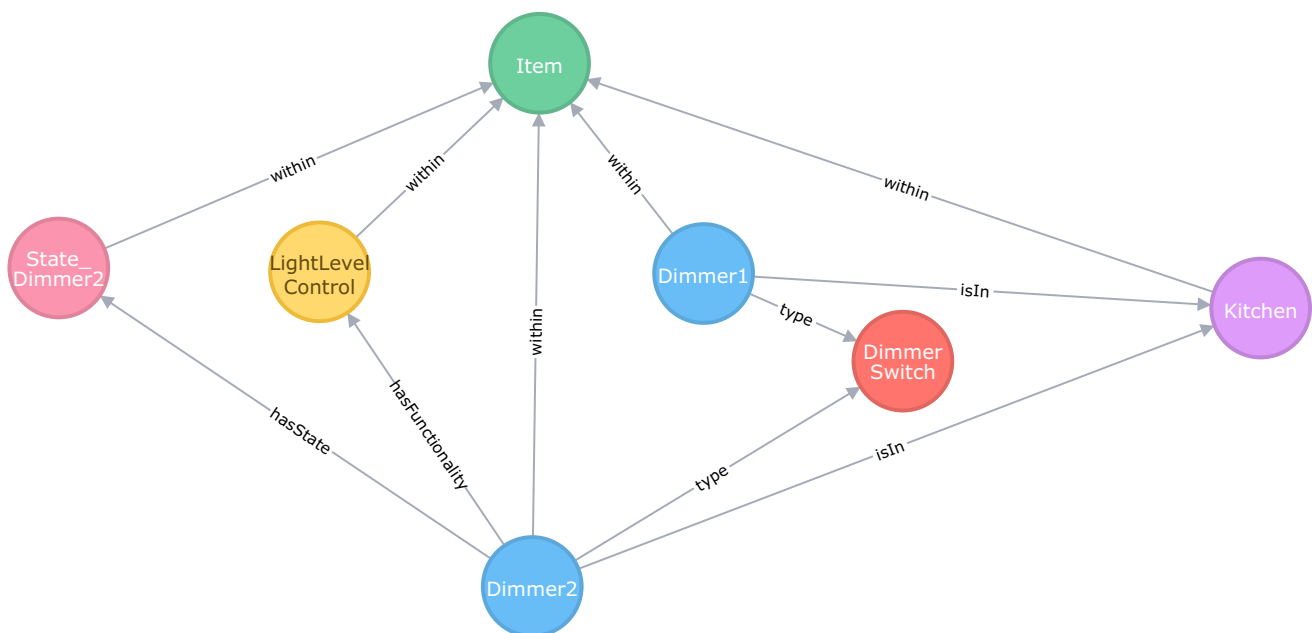


Fig. 5 Parts of the context model for a dimmer switch

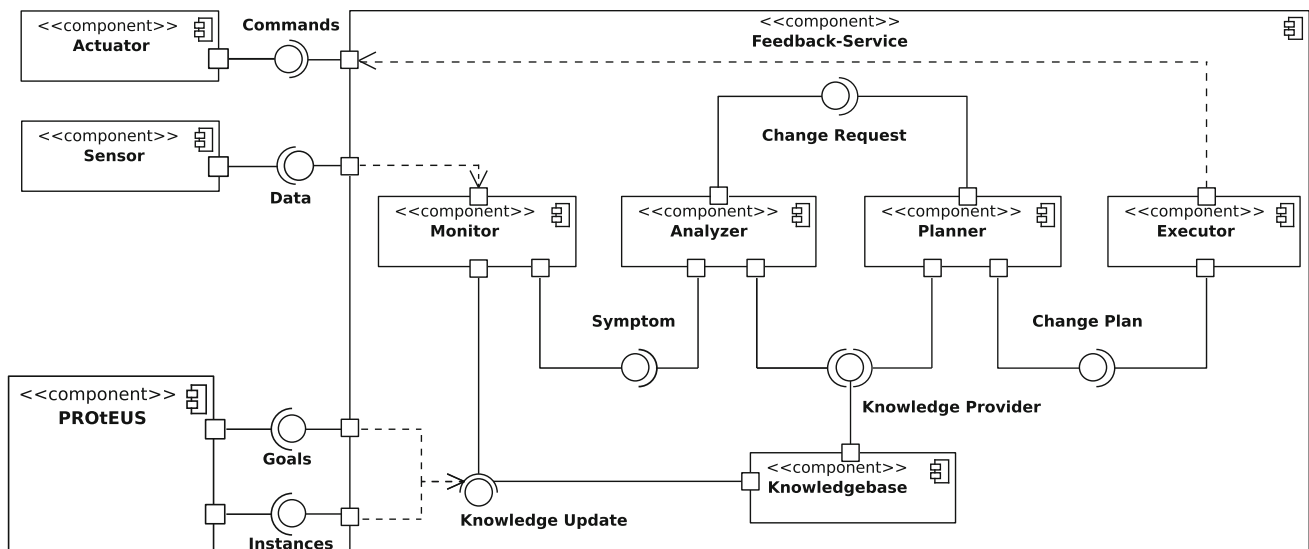


Fig. 6 The MAPE-K feedback loop as a component-based Web service implementation

also part of the model. Dimmer2 is able to control the light levels (*LightLevelControl*), which includes the commands *On*, *Off*, *Up* and *Down* as subclasses for controlling the lights. The knowledge base also contains classes and instance data specific to the aforementioned attributes regarding the MAPE-K feedback loop. It stores process and process step instance data, goals and their objectives to be used within the feedback loop, as well as states and compensation actions executed for the objectives (cf. Fig. 7).

4.4 Consistency levels and ranges

Depending on the application domain, maintaining a strict level of CPC regarding all context factors is not always necessary or feasible. While within smart factories a high level of precision is required to yield a high product quality, context factors in smart home environments can be in a certain range to achieve the desired levels of comfort (e.g., with respect to room temperature or illumination). We support the definition of conditions for satisfied and compensation assertions based on comparison operators to define—besides equality—thresholds and ranges for measurable (numerical) context factors. This is complemented by an optional *Consistency Level*, which can be specified as part of an objective. This level defines the minimal threshold on a relative scale (i.e., 0–100%) that has to be reached for the particular context factor to fulfill the objective. With respect to the lighting scenario the objective “Enough light for working” could already be fulfilled at 90% of the necessary light levels in a smart home and it has to be at 100% in a smart factory. More abstract non-numerical context factors are usually compared based on the equality/comparison function for the respective data type.

4.5 Executing self-adaptive CPS workflows

Our implementation uses the PROtEUS system described in [39] as basic execution environment. Upon execution of a process step, the engine evaluates the *cyberPhysical* flag of the process step in question. In case it is set, a request containing process instance information, the goal and its objectives is sent to the *Feedback Service* shown in Fig. 6 to execute the MAPE-K loop for the process step instance. The process engine then executes the process step in the “regular” way with the Feedback Service running the MAPE-K loop in parallel. The feedback loop is initialized with the goal and objectives of the process step instance to test for *CPC*. The process engine waits for the Feedback Services’ response and continues accordingly (i.e., it resumes the execution if the goal is satisfied or it enters a failure branch). The MAPE-K phases are executed as follows:

4.5.1 Monitor

The *Monitor* component constantly updates the knowledge base by monitoring context data from sensors and other sources. During this phase, the component performs a pre-analysis of the incoming data based on defined thresholds to evaluate the significance of changes within the data. In case there is a significant delta (called a *Symptom* in Fig. 3), the new values—according to the context paths defined in the specific goals—are fed forward into the *Analyzer* component. The OpenHAB middleware for IoT [43] serves as our main source of sensor data. To add data sources a new *Monitoring Agent* has to be implemented connecting the Feedback Service’s monitor and the new source.

4.5.2 Analyze

The *Analyzer* component evaluates the received symptoms with respect to the objectives contained in the process step. If the satisfied assertion can be evaluated as *true*, then the process can be continued. If the compensation assertion is true—e.g., a certain threshold was passed or a timeout was reached—then an inconsistency is assumed and a change request including a description of the mismatch is sent to the *Planner* component. In case the overall goal can be met (i.e., all patterns defined in the satisfied assertions can be detected within the symptoms), the process engine continues with the execution of the consistent process instance. If neither the satisfied assertion nor the compensation assertion can be evaluated as *true*, the feedback loop will be repeated.

4.5.3 Plan

In the *Plan* phase, compensations for the provided mismatch(es) are searched for. The change management of the feedback loop uses the plan request to find an alternative way of reaching the goal [25]. We use the information about sensors and actuators contained in the knowledge base (cf. Sect. 4.3) to find a compensation action automatically by specific graph queries. Sensors and actuators are directly linked to the physical values they measure and manipulate. A *Compensation Query* defines generic criteria for actuators that have to be fulfilled to replace a defective actuator. For our smart home uses cases, we assume that a replacement actuator has to be in the same room (context) as the sensors defined in the context path and its control functionality has to influence the same physical property that the sensor is able to measure. Executing the query on the knowledge base returns available actuators that match these criteria and their commands that are associated with REST services. The *Compensation Query* can be modified with respect to new use cases, criteria and requirements. A *Compensation Repository* containing multiple queries with respect to different error cases is also supported.

The mismatch information currently contains simplified descriptions of the deficits that occurred (e.g., the value in the context path is *too high* or *too low*). The *Planner* component is able to interpret these deficits semantically and to derive change plans specifying which of the determined actuator command needs to be executed by the replacement component. For example, if the luminance values have not reached a certain threshold, an alternative operation for the same actuator or an alternative actuator able to manipulate the same physical value in the same room is selected. Based on the *too low*-mismatch, the *Up* command is chosen for the original actuator or the replacement resource to increase the current luminance values. The derived command includes the

specific REST service address for invocation of this particular actuator service by the Feedback Service.

4.5.4 Execute

The *Executor* component receives the change plan from the planner to execute the derived compensation/change actions on the process resources. In our approach, this means executing the command derived in the planning phase for the alternative actuator (process resource). During execution of the change plan, the MAPE-K loop is used again to check for objective and goal fulfillment (*Cyber-physical Consistency*) with respect to the satisfied assertion of the respective process step. At this point, *Consistency Levels* for the process step can help to reduce the number of necessary MAPE-K loop iterations and therefore to reduce the costs of executing the process step. If all goals can be satisfied by applying the MAPE-K loop, CPC is assumed and regular process execution continues.

In case not all objectives reach the *satisfied* state due to the MAPE-K loop not finding suitable compensations for the occurred errors (i.e., no alternative resources are available), the execution of the process step will reach a failure state and the *Failure Port* along with its modeled failure branch will be activated. In our lighting example, a human task requesting the user to take care of the unresolvable issue is triggered and also analyzed by the MAPE-K loop mechanism—by adding the corresponding CPS attributes to the human task—to see if the luminance level increases and process execution can be continued as planned.

Figure 7 shows the contents of the knowledge base related to the execution of the feedback loop for the smart lighting scenario. The process *LightControl* runs within the *home*. It has the goal “Enough light for working,” which includes the objective “kitchen light intensity > 700 lux within 5 s.” This objective corresponds to the examples in Sect. 4.2. To fulfill the objective the *Up*-command was executed for Dimmer1, which was not able to reach the required light levels. As a compensation found during the Plan phase the *Up*-command was executed for Dimmer2 as alternative resource to fulfill the objective.

4.6 Implementation

The feedback loop is implemented as a component-based Micro-service in Java with interfaces to the physical world in the form of sensors and actuators. It receives goals and instance information about the process (step) to be executed from the process engine (here: *PROtEUS*). Inside the *Feedback Service*, there are components responsible for each of the MAPE phases as well as the *Knowledge Base*. Figure 6 presents the components and interfaces in UML notation. We implemented the *Feedback Loop* as a dedicated REST

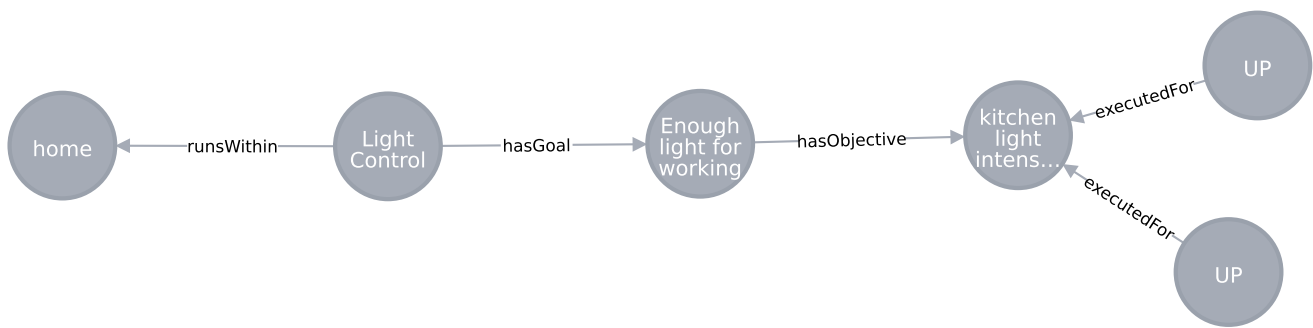


Fig. 7 Goal, objective and commands executed during the feedback loop for the light control process stored in the knowledge base

service that can be invoked by arbitrary process engines and applications.

5 Evaluation

The following experiments in the Smart Home domain were conducted to provide a proof-of-concept evaluation and to illustrate the benefits and limitations of our approach. We used the extended PROtEUS meta-model, editor and engine [39,41] to model and execute the smart lighting process as described in Sect. 2.

5.1 Scenario: smart lighting

The process goal is to ensure optimal lighting conditions (i.e., between 650 and 750 lux luminance) for the work environment in an energy-efficient way. The process steps are embedded into a loop to model a continuous automatic light control.

5.1.1 Experiments

We ran the process in a controlled laboratory environment to monitor the execution and MAPE-K behavior. The setup used one light sensor and two lamps controlled by dimmer power switches. Basic REST services provided by the OpenHAB IoT middleware [19] are used to retrieve sensor values and call actuators from the PROtEUS process engine [39]. To verify the process execution, we ran experiments for three different scenarios (*Baseline*, *MAPE-K+* and *MAPE-K++*).

Baseline The *Baseline* scenario represents the basic process execution not using the MAPE-K loop (i.e., no *cyber-physical* process steps). We execute the process (cf. Fig. 1) consisting of switching on the light and then regulating the light level to keep in balance between 650 and 750 lux by dimming up or down. Once a balance is reached, we switch off the lamp (*Dimmer1*) to simulate a broken light bulb. The process then tries to continuously dim up the light without

reaching the desired levels. Without the feedback of additional sensors, this broken light bulb cannot be recognized by the process engine/control software and the process will continue assuming a wrong state (*CPC violation*).

MAPE-K+ In the *MAPE-K+* scenario, we marked the REST process steps as *cyber-physical* and included goal specifications as described in Sect. 4.2. The goal and its objectives for the *DecreaseLight* step state that the process step execution should lead to a luminance level of at most 750 lux within 5 s. The *IncreaseLight* REST service call is annotated with a goal stating that this process step should lead to light levels over 700 lux within 5 s (cf. Fig. 8). If these goals cannot be fulfilled, the Feedback Service will search for compensation actions to dim up or dim down other light sources in the same context. We made sure that another light source having the same functionality was available such that compensation actions can be found and executed based on the compensation query. In analogy to the *Baseline* experiment we switch off the first light (*Dimmer1*) after the regular process reached a balance. The feedback loop detects this CPC violation, and it restores a consistent state by dimming up the second light source (*Dimmer2*) until the satisfied assertion is fulfilled.

MAPE-K++ We use the *MAPE-K++* scenario to illustrate that the MAPE-K-enabled process has advantages over the baseline process even in case the MAPE-K loop fails and not all objectives can be satisfied. For this scenario we added a *Human Task* to the process, which will be triggered if the feedback loop is not able to restore CPC (cf. Fig. 9). We execute this process in the same way as the *MAPE-K+* runs. After the second lamp restored CPC by dimming up, we switch off this lamp as well. This triggers the feedback loop again, which will reach the “failed” state as it is not able to find any compensations. The respective failure branch is activated and a human task is sent to the resident’s Tablet device instructing him/her to check the lights manually [40]. After fixing the problem the user confirms the completion of this task and the light control loop starts again.

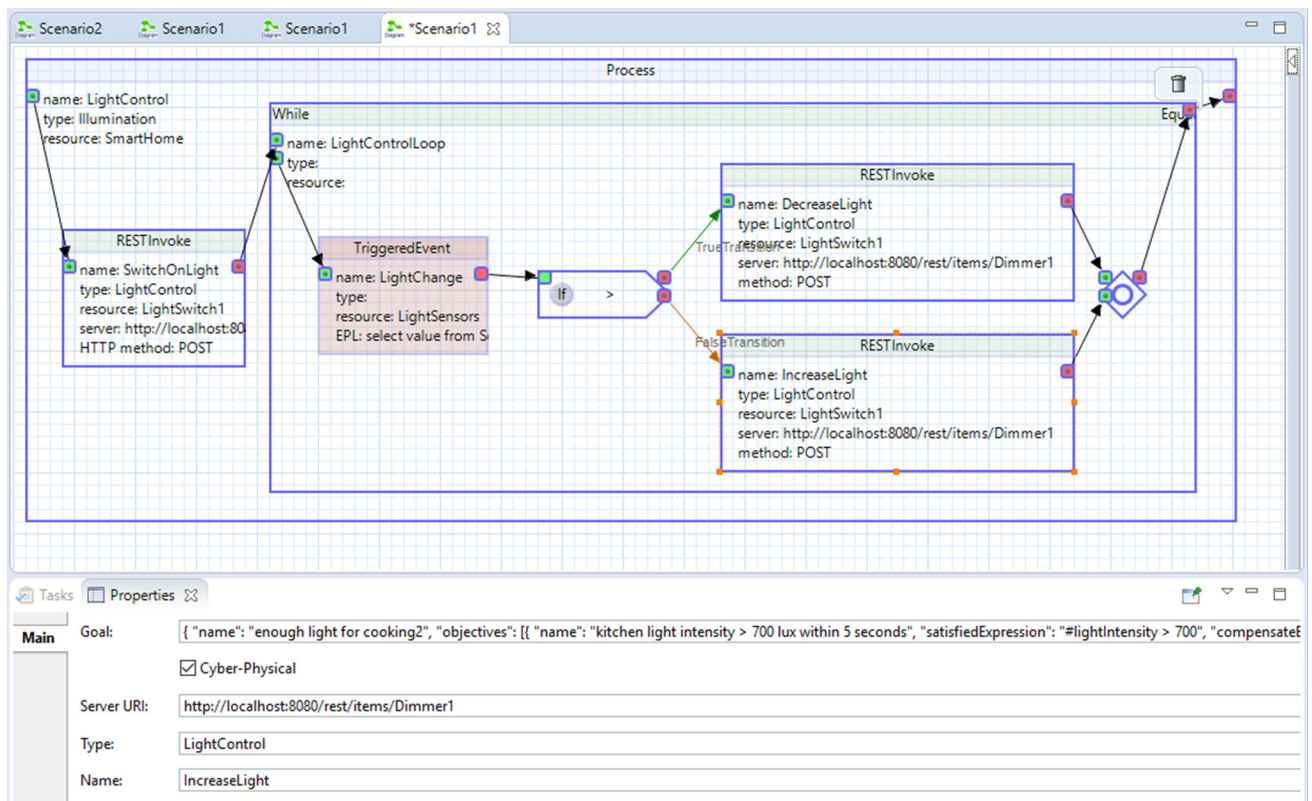


Fig. 8 Process model for the Smart Lighting scenario with MAPE-K attributes in Editor

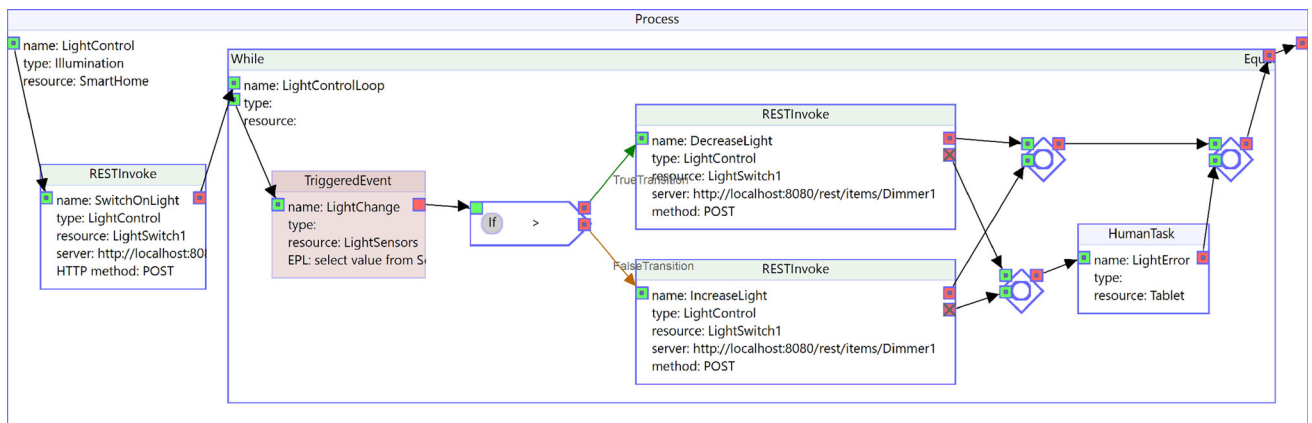


Fig. 9 Process model for the Smart Lighting scenario with MAPE-K and human task

5.1.2 Results

Baseline Figure 10 shows the values of the light sensor (a) and power levels for the dimmer switch (b) for the Baseline experiment over time of day. The corresponding process execution sequences over time are presented in Fig. 10c. As the execution of active process steps (*IncreaseLight*, *DecreaseLight*, *If*) usually takes less than 10 ms, we summarized the execution of one iteration of the *LightControlLoop* subprocess as an *IncreaseLoop* or *DecreaseLoop*. Within one of these

loops the first step is always a *TriggeredEvent* process step whose activation takes a major part of the loop's execution time. This process step is finished if it recognized a certain pattern within the sensors values it is analyzing (i.e., it reacts to changes in the physical environment). It is then followed by an *If* step and an *IncreaseLight* or *DecreaseLight* step in the loop. From the diagrams we see that after switching on the light to 100% power, the process needs some iterations of decreasing and increasing the power level (increases and decreases of approx. 10% per command) to reach a sta-

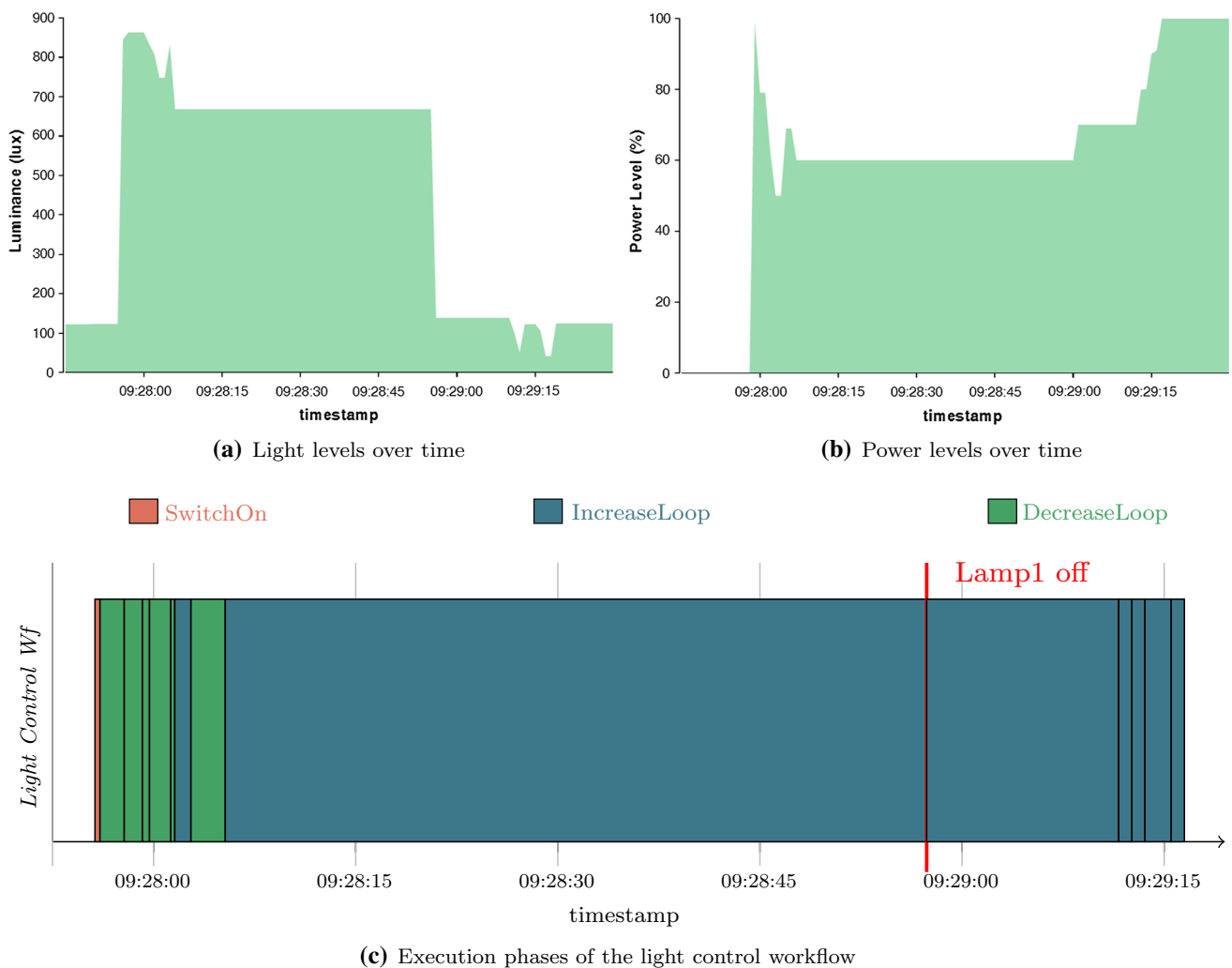


Fig. 10 Light sensor values (a), dimmer switch (b) values and workflow execution (c) for Baseline experiments

ble state for the light levels (between 650 and 700 lux) after approx. 9 s. We then waited for 52 s to switch off the light, which causes a sudden drop within the luminance levels. The *IncreaseLoop* is executed several times to increase the power levels (up to 100%) without reaching the desired light levels. Unless stopped the process would continue indefinitely.

MAPE-K+ Figure 11 shows the values of the light sensor (a) and power levels for the dimmer switches (b) for the MAPE+ experiment over time of day. The corresponding process execution sequences over time are presented in Fig. 11c. Again after switching on the light, the process tries to dim down the lamp to reach the desired light levels. After the first *Decrease* command the feedback service is triggered for the first time to regulate the light within the MAPE phases—eventually reaching a state of balance after a second iteration (approx. 33 s after the process started). We waited 39 s to switch off the first lamp, which leads to a sudden decrease of the light levels. Following the process tries to dim up this lamp again with the Feedback Service checking

for consistency. As the “regular” process step is not successfully increasing the light, the MAPE-K loop starts analyzing relevant sensor data (symptoms) with respect to the compensation and satisfied assertions. Table 1 shows the number of iterations for each phase (*Monitor*, *Analyze*, *Plan*, *Execute*) of this particular execution of the MAPE-K loop. Monitoring and updating the knowledge base is done constantly in parallel with the other phases. From this monitoring data 64 sensor updates were analyzed, 7 of which led to the execution of the planning phase as the compensation assertion was evaluated positively (i.e., the light levels have not reached 700 lux within 5 s after the execute phase). During a planning phase (taking about 1,7 s on average) the second lamp was found and its *Up*-command selected to be executed to increase the light by approx. 10%. After 7 repetitions of executing this command and 30 s in total the feedback service (FB-Service) fulfilled the goal and the process continued. Our implementation of the feedback service relies heavily on asynchronous communication among the phases of the MAPE loop, which

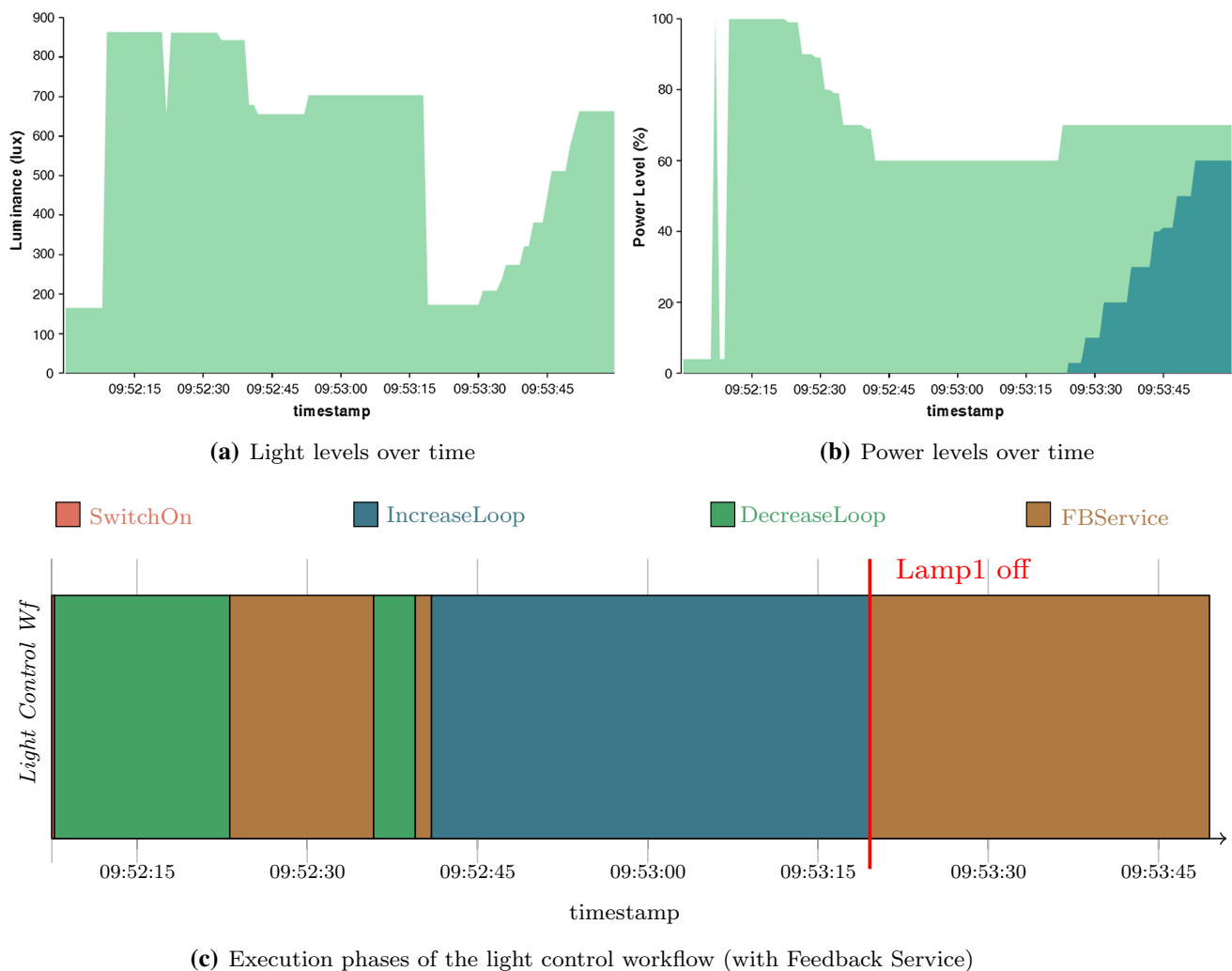


Fig. 11 Light sensor values (a), dimmer switch (b) values and workflow execution (c) for MAPE-K+ experiments

Table 1 Number of iterations for each feedback phase and their average duration for the MAPE-K+ experiment

Metrics\phase	M	A	P	E	FB-service
Iterations (#)	–	64	7	7	1
Duration (ø in ms)	–	4	1699	8	29925

is why a simple summation of the average durations of each phase multiplied by the number of iterations will not yield the overall execution time.

MAPE-K++ Figure 12 shows the values of the light sensor (a) and power levels for the dimmer switches (b) for the MAPE++ experiment over time of day. The corresponding process execution sequences over time are presented in Fig. 12c. The first phases of the process are very similar to the MAPE-K+ experiments: switching on the light; decreasing to reach a balance; increasing with the help of feedback loop to compensate for the failed first lamp. After fulfilling this

goal we waited 17 s to switch off the second lamp, which triggered another *Up*-command and the feedback service trying to compensate for this error. The statistics for this execution of the feedback service are displayed in Table 2. From 107 Analyze phases based on sensor Monitorings 12 Plan phases were initiated trying to compensate for the failed second lamp. Within these planning steps and the following Execution steps, the system tried to dim up all available lamps matching the context criteria to the maximum levels. Compared to the MAPE-K+ experiments the planning phases were significantly faster, because the corresponding graph queries were already known and cached from the previous feedback loop execution (compensating for failed lamp 1). After 38 s the feedback service (FB-Service) reports a failure back to the process engine not being able to find any other compensation actions, which triggers a *Human Task* to be send to a Tablet device. Upon receiving this request, we switch the first lamp back on manually and confirm the human task (taking about 7 s). The light control loop then

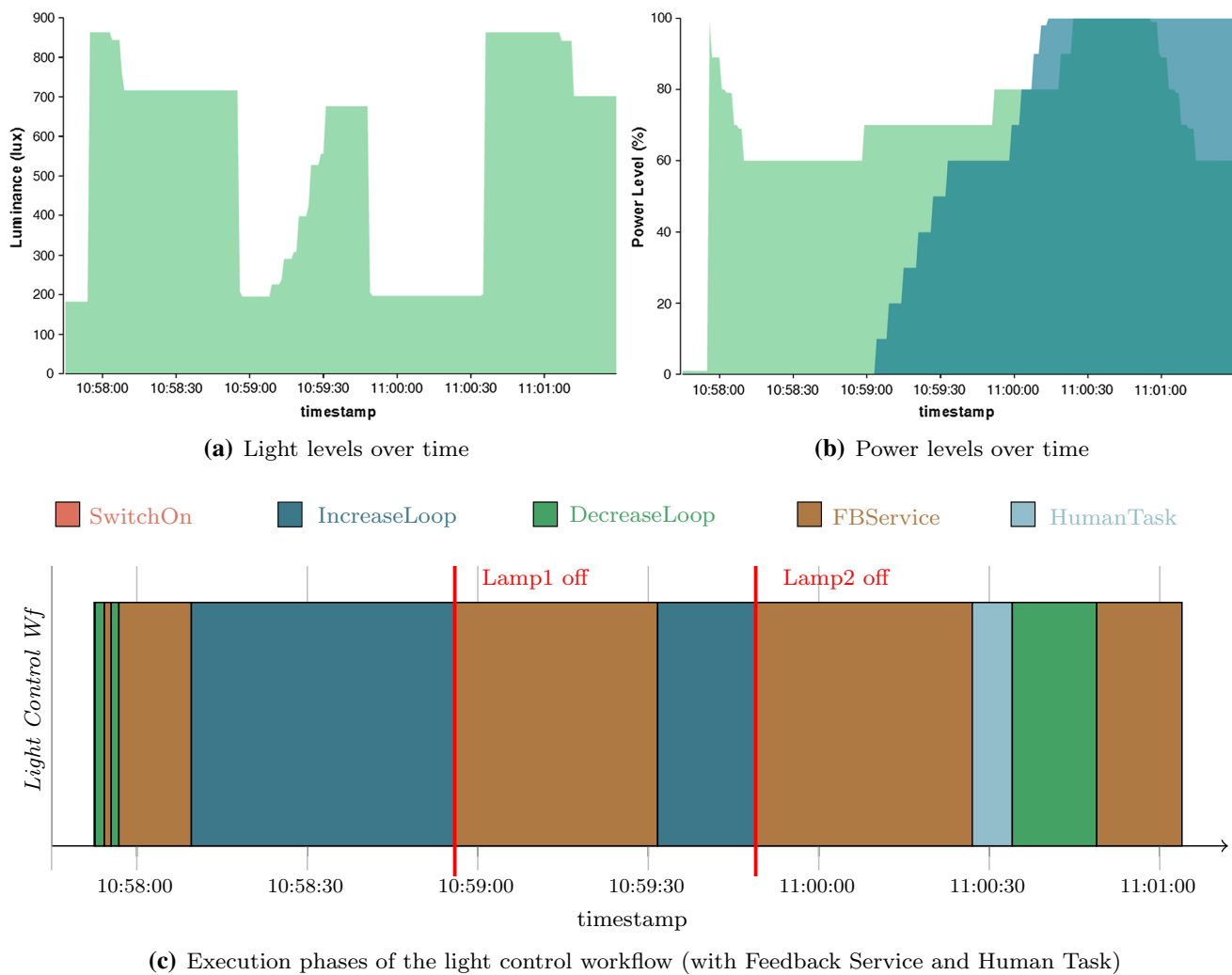


Fig. 12 Light sensor values (a), dimmer switch (b) values and workflow execution (c) for MAPE-K++ experiments

Table 2 Number of iterations for each feedback phase and their average duration for the MAPE-K++ experiment

Metrics\phase	M	A	P	E	FB-service
Iterations (#)	–	107	12	12	1
Duration (ϕ in ms)	–	3	12	2	38100

continues with dimming down lamp 1 again to reach the desired luminance level.

5.2 Discussion

The previous sections show that with the introduction of the new *physical* dimension for business processes, there is a need to also verify the real-world effects of processes in CPS. Feedback loops help to monitor, analyze, plan and execute real-world processes to check for consistency between cyber and physical states at runtime. Sections 4.2–4.5 discussed

extensions for process meta-models and engines. Section 5.1 showed the feasibility of our approach with the help of a proof-of-concept implementation for smart home use cases as examples for CPS.

5.2.1 Process experiments

The experiments show expected behavior of the process execution system and the newly introduced MAPE-K Feedback Service. The execution times of both systems are reasonable and suitable for executing workflows in CPS (despite not guaranteeing real-time behavior, which is out of focus of our work). Our application scenarios are set in the smart home domain, where actions in the physical world (activation of actuators, updates of sensor values) take significantly longer than the processing and computations. With the introduction of the MAPE-K feedback loop based on goals, automated error handling can be outsourced from the process to the feedback service, which controls adaptations to the process

resources trying to reach the defined goal. The feedback loop introduces almost no overhead with respect to the execution times of error-free processes as only data analysis will be performed. If the process goals are already known (cached), then also the planning and execution phases only take a few milliseconds to be executed. By defining a consistency level other than 100% for numerical values the number of MAPE-K iterations and with that the overall execution time can be reduced (*Scalable Consistency*). In general, we observed an increase of successful process executions with the introduction of feedback loops.

5.2.2 Process modeling

Regarding the modeling of cyber-physical processes, we present a generic extension that can be applied to existing meta-models. With the introduction of goals and objectives for formalizing real-world effects of a process, a strict separation of concerns is achieved as the “regular” (cyber) process can be modeled in a known way and the physical aspects are modeled as extensions to that. These extensions assume a composite process structure and can therefore be applied at the process, subprocess and activity level [41]. The more declarative approach of using goals and objectives reduces the overall modeling effort as not every possible error and unintended behavior as well as corresponding failure handling processes have to be modeled explicitly. This is especially relevant for complex CPS with varying contexts and dynamic components resulting in a high level of unanticipated and emergent behavior [23]. However, the modeling of the objectives requires detailed knowledge about contexts, sensors and actuators, and real-world effects of the process. Using more sophisticated approaches for formalizing and deducing this knowledge may further reduce the modeling effort (e.g., as proposed in [28]).

5.2.3 Process execution

The application of the MAPE-K loop for processes can be seen as a fine-grained realization of the BPM lifecycle at runtime [1]. The monitoring and analysis phases correspond to the diagnosis after process enactment, followed by process (re)design and system configuration in the plan phase, and finally the enactment (execute) of the adapted process. The domain, business and process knowledge is contained in the knowledge base. We propose to implement the MAPE-K loop using dedicated components for each phase of the loop. The component-based approach increases modularity and reusability for other processes and applications. Our implementation provides a Web service with standardized interfaces to be used by arbitrary BPM systems, services and applications to apply feedback loops to process-aware information systems. The execution of “regu-

lar” processes is extended indirectly with additional process steps according to the phases of the feedback loop. This way, the processes and BPM systems become more self-aware of their executions and self-adaptive through the feedback loop [6]. However, the application of feedback loops to ensure the outcome of the process execution also introduces additional computational overhead, which may be counter-productive in some CPS domains, e.g., with respect to real-time constraints. With the Feedback Service we add a new software component to workflow management systems, which requires additional resources regarding the operation of and communication with this service. This may not be feasible in resource constraint computing environments.

5.2.4 Process adaptation

The MAPE-K loop can be used as a general framework for enabling self-adaptive workflows that influence physical world context values. In the *Plan* phase of the MAPE-K subprocess, a suitable compensation for the occurred mismatch of cyber and physical world state is searched in the knowledge base derived from criteria defined in the compensation queries. This could be complemented by more sophisticated means for service discovery (e.g. as proposed in [24] or [19]), but this is subject to future work. The process instance is adapted with respect to the process resources and executed afterward, which leads to a higher degree of autonomy for process executions in CPS. In our implementation the adaptation is currently done on the resource instance level as the physical devices and entities in CPS are prone to errors due to fluctuating availability, constraint resources, and other physical failures and inaccuracies. However, the MAPE-K framework is also suitable to adapt business processes on other levels (e.g., data or control flow) and based on other criteria (e.g., QoS or KPI levels) [32]. In addition the automated finding of replacement resources and process adaptations after a violation of cyber-physical consistency can be improved by more sophisticated reasoning and deduction mechanisms (e.g., as proposed in [28] or [47]). In general, various techniques can be plugged in to our component-based framework by programming against the offered interfaces to realize the individual MAPE-K phases.

5.2.5 Process consistency

The verification and matching of the assumed real-world state with the actual physical world effects of an executed process is identified as one of the main challenges for real-world processes [52]. To enable this verification, we introduce the notion of *Cyber-physical Consistency CPC*. The MAPE-K feedback loop provides means for automating the detection of inconsistent states based on sensor and context data and also for restoring consistency to a certain degree (*Consistency*

Level). One of the prerequisites for this approach to work is a stable and resilient sensor infrastructure. We support the use of multiple redundant sensors and averaging/fusing their values as part of the respective Cypher query in the objective definitions. A more detailed discussion of errors in sensor infrastructures correlated with business processes can be found in [51]. Our approach increases resilience against failures, imprecisions and other partially unanticipated situations for process resources that influence the real world. With the MAPE-K feedback loop we can check the outcome of the execution of a process activity if its effects on the environment are known. Based on the mismatch determined by the respective sensor readings (*too low*, *too high* or *unequal*) compensating actions can be derived with the help of the knowledge base to try to correct the error. Eventually, this may lead to an increase of safety and flexibility, reduced resource consumption and an optimization of the overall environment. Despite the goal of reaching a higher level of automation, the user can still be integrated into the workflows in case the MAPE-K loop fails and consistency has to be restored manually.

6 Related work

As many existing “traditional” BPM systems and notations as well as adaptive process management systems are not designed to handle processes in the IoT and CPS, real-world processes have started to become a vibrant field of research within the BPM community [27,30].

The concept of having a virtual model of physical characteristics and entities representing their physical equivalents in CPS is suggested, for example, by de Roo et al. [35] and by Stork [44] in a more general context. To point out a need for also considering real-world effects of processes, Wombacher discusses ways to correlate business workflows and physical objects based on sensor data and process states [51,52]. Various approaches for integrating real-world context data and objects into business processes were proposed with the emergence of smart spaces [3,8,16,31]. In [11] Glombitza et al. show the application of BPEL processes in sensor nodes to process data from IoT devices. Especially the BPMN4CPS extensions suggested by Graja et al. [12] and the uBPMN extension for modeling ubiquitous processes by Yousfi et al. [53] are interesting approaches to integrate cyber-physical components and interactions into business processes. However, they extend BPMN with new CPS specific devices, components and events—not considering the actual effects of the processes in the real world. A framework for collaborative task handling based on processes for IoT is discussed in [21].

Weidlich et al. discuss an approach for optimizing event patterns for process monitoring by using process model

knowledge and event processing [48], which can be applied to automatically adapt the goals and objectives in successive MAPE-K iterations (*Meta-adaptation* [34]). Frincu shows D-OSyRIS as an example of a self-healing workflow engine for distributed process [10]. In [50] and [17] Wieland et al. present SitOPT—a workflow management system that uses situation recognition based on sensor data from production machines to adapt processes in case of errors and execute fault handling templates depending on the specific type of error situation. The SmartPM system by Marella et al. is able to adapt to errors during process execution based on models of expected reality and actual reality and recover from a potential gap between these two worlds using situation calculus and planning [28,29]. Especially the adaptation algorithms are worth to be investigated for integration into the MAPE-K loop. However, these approaches are tied to specific workflow engines and software architectures. Our approach aims at proposing a more general framework for self-management of workflows in CPS.

From the self-adaptive and multi-agent systems communities, there have been several works proposed to combine Deming cycles with BPM systems to deal with failures in the *cyber* world and increase autonomy of BPM systems [2,32]. However, these approaches work on a purely virtual and organizational level and do not consider the issue of cyber-physical synchronization for processes [33]. More general software architectures and frameworks for self-adaptive systems include the MUSIC middleware by Rouvoy et al. [36] and the MORPH reference architecture by Braberman et al. [5]. These approaches provide the general building blocks for the realization of our framework in the context of CPS.

In comparison with the aforementioned work on using BPM technologies to execute (partial) real-world processes, our new approach of combining the worlds of BPM and self-adaptive system in CPS leads to an increased level of autonomy and resilience to failures for cyber-physical workflows. We use the generic idea of the MAPE-K feedback loop as basis for extensions to existing process meta-models and engines, which is why our approach can be regarded as a more general, technology agnostic framework for implementing self-adaptive workflows. Due to the meta-level extensions and component-based approach, arbitrary process notations can be used and various technologies for realizing the MAPE-K phases can be plugged in. Our framework can be used in combination with arbitrary existing service-based BPM systems not requiring a high adjustment effort.

7 Conclusion

In this work, we presented a framework for enabling self-adaptive workflows for cyber-physical systems (CPS). The application of BPM technologies to automate processes in

CPS introduces the new requirement of providing a consistent view of processes' virtual world and real-world effects (*Cyber-physical Consistency*). To achieve this goal, we propose to apply the MAPE-K feedback loop to monitor and analyze real-world process execution using additional sensor and context data and to correlate these data with the process execution. We use graph queries on a knowledge base to find a compensation (replacement resource or command) to be activated in case of inconsistencies and unanticipated errors. In comparison with related work, our approach reaches a high level of autonomy and resilience against failures for physical world process execution due to the capability of self-adaptiveness while still being able to keep the human in the loop. Performance measurements showed the feasibility of our approach in the smart home domain. The model extensions and execution components for the MAPE-K loop can be applied to various process notations and engines, thus resulting in a generic framework for self-adaptive real-world processes. Technologies used in the MAPE-K components can be exchanged and extended easily due to a component-based software architecture. We have shown that the application of feedback loops increases success rates and resilience of workflow executions in CPS.

With respect to future work, we will investigate the application of alternative algorithms in the *Analyze* and *Plan* phases to reduce modeling effort and to increase autonomy. Stream-based mining of processes for compliance checking combined with sensor data may lead to an increased accuracy of determining cyber-physical consistency [27]. We will also apply the generic MAPE-K framework to decentralized process execution environments as proposed in [42] to increase resilience of CPS workflows in the context of distributed processes. An evaluation of the proposed framework for retrofitting of existing non-CPS workflow engines is current work in progress for three open-source process management systems.

Acknowledgements This research has received funding under the Grant Number 100268299 ("CyPhyMan" project) by the European Social Fund (ESF) and the German Federal State of Saxony. Kudos to André Kühnert for helping with the implementations.

References

1. Aalst, W.M.P., Hofstede, A.H.M., Weske, M.: Business Process Management. In: International Conference, BPM 2003 Eindhoven, The Netherlands, June 26–27, 2003 Proceedings, Chapter Business Process Management: A Survey, pp. 1–12. Springer, Berlin (2003)
2. Andonoff, E., Bouaziz, W., Hanachi, C., Bouzguenda, L.: An agent-based model for autonomic coordination of inter-organizational business processes. *Informatica* **20**(3), 323–342 (2009)
3. Baumgrass, A., Di Ciccio, C., Dijkman, R. M., Hewelt, M., Mendling, J., Meyer, A., Wong, T. Y. GET controller and UNICORN: event-driven process execution and monitoring in logistics. In: BPM (Demos), pp. 75–79 (2015)
4. Bonino, D., Corno, F.: Dogont-ontology modeling for intelligent domotic environments. In: Sheth, A., Staab, S., Dean, M., Paolucci, M., Maynard, D., Finin, T., Thirunarayan, K. (eds.) *The Semantic Web-ISWC 2008*, Lecture Notes in Computer Science, vol. 5318, pp. 790–803. Springer, Berlin (2008)
5. Braberman, V., D'Ippolito, N., Kramer, J., Sykes, D., Uchitel, S.: Morph: a reference architecture for configuration and behaviour self-adaptation. In: *Proceedings of the 1st International Workshop on Control Theory for Software Engineering*, pp. 9–16. ACM (2015)
6. Brun, Y., Serugendo, G.D.M., Gacek, C., Giese, H., Kienle, H., Litoiu, M., Müller, H., Pezzè, M., Shaw, M.: Engineering self-adaptive systems through feedback loops. In: *Software engineering for self-adaptive systems*, pp. 48–70. Springer, Berlin (2009)
7. Conti, M., Das, S.K., Bisdikian, C., Kumar, M., Ni, L.M., Passarella, A., Roussos, G., Trster, G., Tsudik, G., Zambonelli, F.: Looking ahead in pervasive computing: challenges and opportunities in the era of cyberphysical convergence. *Pervasive Mob. Comput.* **8**(1), 2–21 (2012)
8. Dar, K., Taherkordi, A., Baraki, H., Eliassen, F., Geihs, K.: A resource oriented integration architecture for the internet of things: a business process perspective. *Pervasive Mob. Comput.* **20**, 145–159 (2015)
9. De Lemos, R., Giese, H., Müller, H.A., Shaw, M., Andersson, J., Litoiu, M., Schmerl, B., Tamura, G., Villegas, N.M., Vogel, T., et al.: *Software Engineering for Self-Adaptive Systems: A Second Research Roadmap*. Springer, Berlin (2013)
10. Frincu, M.E.: D-OSyRIS: a self-healing distributed workflow engine. In: *International Symposium on Parallel and Distributed Computing*, pp. 215–222 (2011)
11. Glombitza, N., Ebers, S., Pfisterer, D., Fischer, S.: Using BPEL to realize business processes for an internet of things. In: *International Conference on Ad-Hoc Networks and Wireless*, pp. 294–307. Springer (2011)
12. Graja, I., Kallel, S., Guermouche, N., Kacem, A.H.: BPMN4CPS: A BPMN extension for modeling cyber-physical systems. In: *2016 IEEE 25th International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE)*, pp. 152–157 (2016)
13. Gray, J., Reuter, A.: *Transaction Processing: Concepts and Techniques*. Elsevier, Amsterdam (1992)
14. Guinard, D., Ion, I., Mayer, S.: In search of an internet of things service architecture: Rest or ws-*? A developers perspective. In: *International Conference on Mobile and Ubiquitous Systems: Computing, Networking, and Services*, pp. 326–337. Springer (2011)
15. Gurgun, L., Gunalp, O., Benazzouz, Y., Gallissot, M.: Self-aware cyber-physical systems and applications in smart buildings and cities. In: *Proceedings of the Conference on Design, Automation and Test in Europe (DATE '13)*, pp. 1149–1154. EDA Consortium, San Jose (2013)
16. Herzberg, N., Meyer, a., Weske, M.: An event processing platform for business process management. In: *17th IEEE International Enterprise Distributed Object Computing Conference*, pp. 107–116 (2013)
17. Hirmer, P., Wieland, M., Schwarz, H., Mitschang, B., Breitenbücher, U., Sáez, S.G., Leymann, F.: Situation recognition and handling based on executing situation templates and situation-aware workflows. *Computing* **99**(2), 1–19 (2016)
18. Huber, S., Seiger, R., Kühnert, A., Schlegel, T.: A context-adaptive workflow engine for humans, things and services. In: *Proceedings of the 2016 ACM International Joint Conference on Pervasive and Ubiquitous Computing: Adjunct (UbiComp '16)*, pp. 285–288. ACM, New York (2016)
19. Huber, S., Seiger, R., Schlegel, T.: Using semantic queries to enable dynamic service invocation for processes in the internet of things.

- In: 2016 IEEE International Conference on Semantic Computing (ICSC), pp. 214–221 (2016)
20. Kephart, J., Kephart, J., Chess, D., Boutilier, C., Das, R., Kephart, J.O., Walsh, W.E.: An architectural blueprint for autonomic computing. In: IBM (2003)
 21. Kim, M., Ahn, H., Kim, K.P.: Process-aware internet of things: a conceptual extension of the internet of things framework and architecture. *KSII Trans. Internet Inf. Syst.* **10**(8), 4008–4022 (2016)
 22. Koetter, F., Kochanowski, M.: Business Information Systems. In: 15th International Conference, BIS 2012, Vilnius, Lithuania, May 21–23, 2012. Proceedings, Chapter Goal-Oriented Model-Driven Business Process Monitoring Using ProGoalML, pp. 72–83. Springer, Berlin (2012)
 23. Kopetz, H.: System-of-systems complexity. arXiv preprint [arXiv:1311.3629](https://arxiv.org/abs/1311.3629) (2013)
 24. Kourtesis, D., Paraskakis, I.: Combining SAWSDL, OWL-DL and UDDI for semantically enhanced web service discovery. *Semant. Web Res. Appl.* 614–628 (2008)
 25. Kramer, J., Magee, J.: Self-managed systems: an architectural challenge. In: Future of Software Engineering (FOSE'07), pp. 259–268. IEEE (2007)
 26. Lee, E.: Cyber physical systems: design challenges. In: 2008 11th IEEE International Symposium on Object Oriented Real-Time Distributed Computing (ISORC), pp. 363–369 (2008)
 27. Leotta, F., Mecella, M., Mendling, J.: Applying process mining to smart spaces: perspectives and research challenges. In: Advanced Information Systems Engineering Workshops, pp. 298–304. Springer (2015)
 28. Marrella, A., Mecella, M., Sardina, S.: SmartPM: an adaptive process management system through situation calculus, indigolog, and classical planning. In: Principles of Knowledge Representation and Reasoning, pp. 1–10. AAAI Press, Menlo Park (2014)
 29. Marrella, A., Mecella, M., Sardina, S.: Intelligent process adaptation in the SmartPM system. *ACM Trans. Intell. Syst. Technol.* **8**(2), 25:1–25:43 (2016)
 30. Meyer, S., Ruppen, A., Hilty, L.: The things of the internet of things in BPMN. In: Advanced Information Systems Engineering Workshops, pp. 285–297 (2015)
 31. Meyer, S., Ruppen, A., Magerkurth, C.: Internet of things-aware process modeling. In: Integrating IoT devices as business process resources. Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 7908 LNCS, pp. 84–98 (2013)
 32. Oliveira, K., Castro, J., España, S., Pastor, O.: Multi-level autonomic business process management. *Enterp. Bus. Process Inf. Syst. Model.* 184–198 (2013)
 33. Perrin, O., Godart, C.: A model to support collaborative work in virtual enterprises. *Data Knowl. Eng.* **50**(1), 63–86 (2004). (advances in business process management)
 34. Piechnick, C., Richly, S., Kühn, T., Götz, S., Püschel, G., Aßmann, U.: Contextpoint: an architecture for extrinsic meta-adaptation in smart environments. In: Sixth International Conference on Adaptive and Self-adaptive Systems and Applications, pp. 121–128 (2014)
 35. de Roo, A., Sozer, H., Aksit, M.: Runtime verification of domain-specific models of physical characteristics in control software. In: 2011 Fifth International Conference on Secure Software Integration and Reliability Improvement, pp. 41–50 (2011)
 36. Rouvoy, R., Barone, P., Ding, Y., Eliassen, F., Hallsteinsen, S., Lorenzo, J., Mamelli, A., Scholz, U.: Music: middleware support for self-adaptation in ubiquitous and service-oriented environments. In: Software engineering for self-adaptive systems, pp. 164–182. Springer (2009)
 37. Saidani, O., Rolland, C., Nurcan, S.: Towards a generic context model for BPM. In: 2015 48th Hawaii International Conference on System Sciences (HICSS), pp. 4120–4129 (2015)
 38. Seiger, R., Huber, S., Heisig, P., Assmann, U.: Enabling Self-adaptive Workflows for Cyber-physical Systems, pp. 3–17. Springer, Berlin (2016)
 39. Seiger, R., Huber, S., Schlegel, T.: Proteus: an integrated system for process execution in cyber-physical systems. In: Gaaloul, K., Schmidt, R., Nurcan, S., Guerreiro, S., Ma, Q. (eds.) Enterprise, Business-Process and Information Systems Modeling, Lecture Notes in Business Information Processing, vol. 214, pp. 265–280 (2015)
 40. Seiger, R., Huber, S., Schlegel, T.: Toward an execution system for self-healing workflows in cyber-physical systems. *Softw. Syst. Model.* 1–22 (2016) (**special section paper**)
 41. Seiger, R., Keller, C., Niebling, F., Schlegel, T.: Modelling complex and flexible processes for smart cyber-physical environments. *J. Comput. Sci.* **10**, 137–148 (2015)
 42. Seiger, R., Niebling, F., Schlegel, T.: A distributed execution environment enabling resilient processes for ubiquitous systems. In: 2014 IEEE International Conference on Pervasive Computing and Communications Workshops (PERCOM Workshops), pp. 220–223 (2014)
 43. Smirek, L., Zimmermann, G., Ziegler, D.: Towards universally usable smart homes-how can MyUI, URC and openHAB contribute to an adaptive user interface platform. In: IARIA, Nice, France, pp. 29–38 (2014)
 44. Stork, A.: Visual computing challenges of advanced manufacturing and industrie 4.0 [guest editors' introduction]. *IEEE Comput. Graphics Appl.* **35**(2), 21–25 (2015)
 45. Talcott, C.: Cyber-physical systems and events. In: Software-Intensive Systems and New Computing Paradigms, pp. 101–115. Springer (2008)
 46. Webber, J.: A programmatic introduction to neo4j. In: Proceedings of the 3rd Annual Conference on Systems, Programming, and Applications: Software for Humanity, pp. 217–218. ACM (2012)
 47. Weber, B., Rinderle, S., Wild, W., Reichert, M.: Case-Based Reasoning Research and Development. In: 6th International Conference on Case-Based Reasoning, ICCBR 2005, Chicago, IL, USA, August 23–26, 2005. Proceedings, chap. CCBR-Driven Business Process Evolution, pp. 610–624. Springer, Berlin (2005)
 48. Weidlich, M., Ziekow, H., Gal, A., Mendling, J., Weske, M.: Optimising event pattern matching using business process models. *IEEE Trans. Knowl. Data Eng.* **26**(11), 2759–2773 (2014)
 49. White, S.A.: BPMN Modeling and Reference Guide: Understanding and Using BPMN. Future Strategies Inc., New York (2008)
 50. Wieland, M., Schwarz, H., Breitenbucher, U., Leymann, F.: Towards situation-aware adaptive workflows: SitOPT—a general purpose situation-aware workflow management system. In: 2015 IEEE International Conference on Pervasive Computing and Communication Workshops (PerCom Workshops), pp. 32–37. IEEE (2015)
 51. Wombacher, A.: A-posteriori detection of sensor infrastructure errors in correlated sensor data and business workflows. In: Proceedings of the 9th International Conference on Business Process Management (BPM'11), pp. 329–344. Springer, Heidelberg (2011)
 52. Wombacher, A.: How physical objects and business workflows can be correlated. In: Proceedings of the 2011 IEEE International Conference on Services Computing (SCC 2011), pp. 226–233 (2011)
 53. Yousfi, A., Bauer, C., Saidi, R., Dey, A.K.: uBPMN: A BPMN extension for modeling ubiquitous business processes. *Inf. Softw. Technol.* **74**, 55–68 (2016)



Ronny Seiger received his Diploma in computer science from Technische Universität Dresden in 2011. He has been a Ph.D. student and member of the Software Engineering of Ubiquitous Systems group from 2012 to 2015. Since 2015 he is a research assistant at the Software Technology group at TU Dresden. His research interests include workflows and processes, Cyber-physical Systems, Internet of Things, robotics, distributed systems and software engineering. His main

focus is on applying workflow technologies for automating processes in Cyber-physical systems.



Steffen Huber is a researcher at the Institute of Machine Tools and Control Engineering at the Technical University of Dresden since March 2017. He is investigating the modeling of reusable knowledge discovery patterns in manufacturing by combining data-intensive and context-sensitive workflows with machine learning algorithms. Before, he was a research assistant at the Junior Professorship in Software Engineering of Ubiquitous Systems within the DFG Research Training

Group RoSI from November 2013 to December 2016. He received his Diploma in Computer Science from the Technical University of Dresden in October 2013. His research interests include context-sensitive workflows, role-based modeling, semantic technologies, Cyber-physical Systems, Internet of Things, Machine Learning, and sensor data analysis.



Peter Heisig is a research associate of the software technology group at Technische Universität Dresden. His research interests include software architectures for self-adaptive distributed systems, cyber-physical systems and the internet of things. Modern web technologies, self-contained systems, programming languages, model-driven design and test-driven development are his major fields of interest. He is passionate about open-source software and contributed to several projects.



Uwe Aßmann is head of the Software Technology Group of the Technische Universität Dresden. Uwe Aßmann got his Ph.D. in 1995 at the Universität Karlsruhe on the generation of program optimizers from graph rewrite systems. Besides invasive model- and software composition, Uwe Aßmann works in the areas of self-adaptive architectures and software systems. He is a member of the IFIP Working Group 2.4 (System Implementation Languages) and member of the

Gesellschaft für Informatik (GI). He is member of the scientific advisor board for Silicon Saxony in the area of “Software,” one of the founding members of the “Cyber-Physical Systems Working Group” of Silicon Saxony, member of the working group “Microelectronics” of the state of Saxony. Uwe Aßmann is also reviewer of the European Union for “Complex Systems.”